

19. Java

```
public class Pedido {
    private Cliente cliente;

    public double calcularDescuento() {
        if (cliente.getHistorialCompras().size() > 10) {
            return 0.15; // VIP
        }
        if (cliente.getHistorialCompras().stream()
            .filter(c -> c.getFecha().after(nuevaFecha(-365)))
            .count() > 5) {
            return 0.10; // Frecuente
        }
        if (cliente.getPerfil().getEdad() > 65) {
            return 0.05; // Senior
        }
        if (cliente.getDatosFiscales().getEstrato() < 2) {
            return 0.20; // Subsidio
        }
        return 0;
    }

    private Date nuevaFecha(int dias) {
        Calendar cal = Calendar.getInstance();
        cal.add(Calendar.DAY_OF_YEAR, dias);
        return cal.getTime();
    }

    public boolean validarCredito() {
        return cliente.getHistorialPagos().stream()
            .allMatch(p -> p.isPagado() &&
                p.getMonto() <= cliente.getLimiteCredito());
    }
}

class Cliente {
    private List<Compra> historialCompras;
    private Perfil perfil;
```

```
private DatosFiscales datosFiscales;  
private List<Pago> historialPagos;  
}
```

20. C#

```
public class SesionManager {
    public class Sesion {
        private Usuario usuarioTemp;
        private DateTime ultimaValidacion;
    }

    public bool login(string email, string password) {
        Sesion sesion = new Sesion();
        Usuario usuario = _repo.validar(email, password);

        sesion.usuarioTemp = usuario;
        sesion.ultimaValidacion = DateTime.Now;

        return _validador.validarSesionTemporal(sesion);
    }
}

public class ValidadorSesion {
    public bool validarSesionTemporal(SesionManager.Sesion s) {
        if (s.usuarioTemp == null) return false;
        if ((DateTime.Now - s.ultimaValidacion).TotalSeconds > 30) {
            s.usuarioTemp = null; // MODIFICA estado ajeno
            return false;
        }
        return true;
    }
}

public class ProcesadorPedido {
    public bool procesar(SesionManager.Sesion s) {
        return s.usuarioTemp != null; // NULL en runtime
    }
}
```

21. Go

```
type MySQLDatabase struct {
    host string
    port int
}

func (m *MySQLDatabase) Connect() error {
    fmt.Printf("Conectando MySQL %s:%d\n", m.host, m.port)
    return nil
}

func (m *MySQLDatabase) Query(sql string) ([]byte, error) {
    // 20 líneas query MySQL
    return []byte("MySQL result"), nil
}

type MongoDBDatabase struct {
    uri string
}

func (m *MongoDatabase) Open() error {
    fmt.Println("Abriendo MongoDB", m.uri)
    return nil
}

func (m *MongoDatabase) Find(collection, query string) ([]byte, error) {
    // 15 líneas Mongo query
    return []byte("Mongo result"), nil
}

type DatabaseProcessor struct {
    db interface{}
}

func (p *DatabaseProcessor) Process() {
}
```

22. Kotlin

```
class UsuarioController {
    private val service = UsuarioService()
    private val repo = UsuarioRepository()

    fun obtenerUsuario(id: Long) = service.obtenerUsuario(id)
    fun crearUsuario(data: UsuarioData) = service.crearUsuario(data)
    fun actualizarUsuario(id: Long, data: UsuarioData) =
        service.actualizarUsuario(id, data)
    fun eliminarUsuario(id: Long) = service.eliminarUsuario(id)
    fun listarUsuarios() = service.listarUsuarios()
    fun buscarUsuario(email: String) = service.buscarUsuario(email)
    fun validarUsuario(id: Long) = service.validarUsuario(id)
    fun bloquearUsuario(id: Long) = service.bloquearUsuario(id)
}

class UsuarioService {
    private val repo = UsuarioRepository()

    fun obtenerUsuario(id: Long) = repo.findById(id)
    fun crearUsuario(data: UsuarioData) = repo.save(data)
}

class UsuarioRepository {
    fun findById(id: Long): Usuario? = null
    fun save(data: UsuarioData): Usuario = Usuario()
}
```

23. Ruby

```
class Pedido # 800 LÍNEAS TOTALES
  attr_accessor :items, :cliente, :descuento, :impuestos, :total

  def initialize(cliente)
    @cliente = cliente
    @items = []
    @descuento = 0
    @impuestos = {}
  end

  def agregar_item(producto, cantidad)
    @items << {producto: producto, cantidad: cantidad}
  end

  def calcular_subtotal
    @items.sum { |i| i[:producto].precio * i[:cantidad] }
  end

  def aplicar_descuentos
    # 100 líneas lógica descuento compleja
    if @cliente.tipo == 'VIP' && subtotal > 100000
      @descuento = subtotal * 0.15
    elsif @cliente.historial.count > 10
      @descuento = 5000
    end
  end

  def calcular_impuestos
    # 120 líneas IVA por país
    @impuestos['iva'] = subtotal * 0.19
    @impuestos['ica'] = subtotal * 0.02
  end

  def generar_pdf # 150 líneas
  def enviar_email # 80 líneas
  def actualizar_stock # 90 líneas
  def validar_credito # 70 líneas
  # TOTAL: 70% DEL PROYECTO AQUÍ
```

end

24. C++

```
class Vehiculo { virtual void acelerar() { std::cout << "Vehículo"; } };
class Motorizado : public Vehiculo {
    void acelerar() override { Vehiculo::acelerar(); std::cout << " motor"; }
};
class Terrestre : public Motorizado {
    void acelerar() override { Motorizado::acelerar(); std::cout << " tierra"; }
};
class Acuatico : public Terrestre { // ;ANIDAMIENTO LOCO!
    void acelerar() override { Terrestre::acelerar(); std::cout << " agua"; }
};
class Anfibio : public Acuatico {
    void acelerar() override {
        if (en_agua) Acuatico::acelerar();
        else Terrestre::acelerar();
    }
};
```


25. TypeScript

```
class Usuario {
    constructor(public nombre: string, public email: string) {}
    getNombre() { return this.nombre; }
    getEmail() { return this.email; }
}

class Producto {
    constructor(public nombre: string, public precio: number) {}
}

class PedidoService {
    procesarPedido(usuario: Usuario, productos: Producto[]): Pedido {
        let subtotal = 0;
        for (let prod of productos) {
            subtotal += prod.getPrecio() * 1;
        }

        if (usuario.getNombre().length > 10) {
            subtotal *= 0.9;
        }

        return new Pedido(subtotal);
    }
}
```